Constraint-Based Problem Solving

Exam scheduling is a constraint-based problem because courses that share students cannot be examined at the same time. Here, a university must assign five courses to three exam slots while respecting every enrollment conflict. This can be modelled as a Constraint Satisfaction Problem (CSP): the courses are the variables, the exam slots are their possible values, and each pair of conflicting courses must receive different slots. Backtracking Search and Forward Checking can then be used to find a valid, conflict-free timetable.

Slot convention: Slot1, Slot2 and Slot3 are the three available exam days. Variables are assigned in the order A, B, C, D, E, and the slots are tried in the order Slot1, Slot2, Slot3.

A. PEAS Description

Component	Description
Performance measure	Produce a complete timetable for all five courses in which no conflicting pair shares a slot.
Environment	Five courses, three exam slots, and the given list of student-enrollment conflicts.
Actuators	Assign or change a course's exam slot; output the completed timetable.
Sensors	Read the courses, the available slots, the conflict list, and the current assignments or remaining domains.

B. Best-Suited Agent Type

A **goal-based agent** is the best fit. Its goal is a complete timetable that satisfies every conflict constraint, and it can use CSP search to assign slots and reject any assignment that breaks a constraint.

The agent also keeps track of the current assignments and the remaining legal slots during the search. This internal state supports its planning, but the main purpose of the task is still to reach the goal of a valid timetable.

C. CSP Formulation and Constraint Graph

Variables: each course is a variable, X = {A, B, C, D, E}.

Domains: every course may take any of the three slots: D(A) = D(B) = D(C) = D(D) = D(E) = {Slot1, Slot2, Slot3}.

Constraints: every conflicting pair must be assigned different slots:

A != B, A != C, B != C, B != D, C != D, C != E, D != E

Constraint graph:

• Vertices: A, B, C, D, E

• Edges: AB, AC, BC, BD, CD, CE, DE

Each edge means that its two courses cannot share a slot.

D. Plain Backtracking Search

Plain backtracking assigns the variables in the fixed order A, B, C, D, E. For each variable it tests the slots in the fixed order and rejects any slot that conflicts with an already assigned neighbouring course.

Step	Variable	Slot tested	Result
1	A	Slot1	Assign
2	B	Slot1	Reject (conflicts with A)
3	B	Slot2	Assign
4	C	Slot1	Reject (conflicts with A)
5	C	Slot2	Reject (conflicts with B)
6	C	Slot3	Assign
7	D	Slot1	Assign
8	E	Slot1	Reject (conflicts with D)
9	E	Slot2	Assign

Final timetable:

Course	Assigned slot
A - Artificial Intelligence	Slot1
B - Database Systems	Slot2
C - Computer Networks	Slot3
D - Operating Systems	Slot1
E - Software Engineering	Slot2

E. Backtracking with Forward Checking

After each assignment, forward checking removes the chosen slot from the domains of the unassigned courses that conflict with the course just assigned. The variable order and value order stay the same. A value that has already been pruned from a domain is skipped instead of being tested as a new assignment.

At the start, every course has the domain {Slot1, Slot2, Slot3}.

Step	Assignment	Domain changes after forward checking
1	A = Slot1	Remove Slot1 from B and C. B = {Slot2, Slot3}; C = {Slot2, Slot3}; D = {Slot1, Slot2, Slot3}; E = {Slot1, Slot2, Slot3}.
2	B = Slot2	Slot1 was already pruned from B, so it is skipped. Remove Slot2 from C and D. C = {Slot3}; D = {Slot1, Slot3}; E = {Slot1, Slot2, Slot3}.

3	C = Slot3	Remove Slot3 from D and E. D = {Slot1}; E = {Slot1, Slot2}.
4	D = Slot1	Remove Slot1 from E. E = {Slot2}.
5	E = Slot2	E's only remaining value is assigned. All courses are now assigned.

No domain becomes empty, so no backtracking is needed.

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

Assignment attempts: 5 actual assignments. Values removed by forward checking are skipped; for example, B = Slot1 is pruned right after A = Slot1 is assigned.

Backtracks: 0.

F. Plain Backtracking vs. Forward Checking

Measure	Plain backtracking	Backtracking with forward checking
Assignment attempts	9	5
Backtracks	0	0
Constraint checking	Checks a value against the current assignments only when it is tried	Also removes incompatible values from the domains of future courses

PYTHON CODE:

```
courses = ["A", "B", "C", "D", "E"]
slots = ["Slot1", "Slot2", "Slot3"]

course_names = {
 "A": "Artificial Intelligence",
 "B": "Database Systems",
 "C": "Computer Networks",
 "D": "Operating Systems",
 "E": "Software Engineering",
}

# Each pair in this list must be assigned different
slots. conflicts = [
 ("A", "B"),
 ("A", "C"),
 ("B", "C"),
 ("B", "D"),
 ("C", "D"),
 ("C", "E"),
 ("D", "E"),
]

neighbors = {course: set() for course in courses}
for first, second in conflicts:
 neighbors[first].add(second)
 neighbors[second].add(first)


def is_consistent(course, slot, assignment):
 """Return True if this slot does not conflict with assigned neighbors."""
for neighbor in neighbors[course]:
 if assignment.get(neighbor) == slot:
 return False
 return True


def plain_backtracking():
 assignment = {}
 attempts = 0
```

```
 backtracks = 0

 def search(index):
 nonlocal attempts, backtracks

 if index == len(courses):
 return True

 course = courses[index]

 for slot in slots:
 attempts += 1
 print(f"Try {course} = {slot}")

 if is_consistent(course, slot, assignment):
 assignment[course] = slot
 print(f" Assign {course} = {slot}")

 if search(index + 1):
 return True

 # This runs only if a later choice leads to a dead end.  print(f"
Backtrack: undo {course} = {slot}")  del assignment[course]
 backtracks += 1
 else:
 print(f" Reject {course} = {slot} (conflict)")  return False

 solved = search(0)
 return solved, assignment, attempts, backtracks


def forward_checking():
 assignment = {}
 domains = {course: slots.copy() for course in courses}
attempts = 0
 backtracks = 0

 def search(index):
 nonlocal attempts, backtracks

 if index == len(courses):
 return True

 course = courses[index]

 # Use the original slot order, skipping values pruned from the domain.  for
slot in slots:
 if slot not in domains[course]:
 print(f"Skip {course} = {slot} (pruned from domain)")  continue

 attempts += 1
 print(f"Try {course} = {slot}")
 assignment[course] = slot
 print(f" Assign {course} = {slot}")

 # Save domains so they can be restored if this choice fails.
saved_domains = {
 variable: domain.copy()
 for variable, domain in domains.items()
 }

 domain_wipeout = False

 # Remove this slot from unassigned neighboring courses.  for
neighbor in neighbors[course]:
 if neighbor not in assignment and slot in domains[neighbor]:
domains[neighbor].remove(slot)
 print(
 f" Forward check: remove {slot} from {neighbor}; "
                                 {domains[neighbor]}"
 )
f"{neighbor} domain =


 if not domains[neighbor]:
 print(f" Domain wipeout for {neighbor}")  domain_wipeout = True
 break

 if not domain_wipeout and search(index + 1):  return
True

 print(f" Backtrack: undo {course} = {slot}")  del
assignment[course]
 domains.clear()
 domains.update(saved_domains)
 backtracks += 1
```

```
    return False

    solved = search(0)
    return solved, assignment, attempts, backtracks


def print_result(title, result):
    solved, assignment, attempts, backtracks = result

    print(f"\n{title}")
    print("=" * len(title))

    if not solved:
        print("No valid timetable found.")
    else:
        for course in courses:
            print(
                f"{course} - {course_names[course]}: "
                f"{assignment[course]}"
            )

        print("Assignment attempts:", attempts)
        print("Backtracks:", backtracks)


print("PLAIN BACKTRACKING SEARCH")
plain_result = plain_backtracking()
print_result("Plain Backtracking Result", plain_result)

print("\n\nBACKTRACKING WITH FORWARD CHECKING")
fc_result = forward_checking()
print_result("Forward Checking Result", fc_result)
```

OUTPUT:

```
PLAIN BACKTRACKING SEARCH
Try A = Slot1
  Assign A = Slot1
Try B = Slot1
  Reject B = Slot1 (conflict)
Try B = Slot2
  Assign B = Slot2
Try C = Slot1
  Reject C = Slot1 (conflict)
Try C = Slot2
  Reject C = Slot2 (conflict)
Try C = Slot3
  Assign C = Slot3
Try D = Slot1
  Assign D = Slot1
Try E = Slot1
  Reject E = Slot1 (conflict)
Try E = Slot2
  Assign E = Slot2

Plain Backtracking Result
=========================
A - Artificial Intelligence: Slot1
B - Database Systems: Slot2
C - Computer Networks: Slot3
D - Operating Systems: Slot1
E - Software Engineering: Slot2
Assignment attempts: 9
Backtracks: 0


BACKTRACKING WITH FORWARD CHECKING
Try A = Slot1
  Assign A = Slot1
  Forward check: remove Slot1 from C; C domain = ['Slot2', 'Slot3']
  Forward check: remove Slot1 from B; B domain = ['Slot2', 'Slot3']
Skip B = Slot1 (pruned from domain)
Try B = Slot2
  Assign B = Slot2
  Forward check: remove Slot2 from C; C domain = ['Slot3']
  Forward check: remove Slot2 from D; D domain = ['Slot1', 'Slot3']
Skip C = Slot1 (pruned from domain)
Skip C = Slot2 (pruned from domain)
Try C = Slot3
  Assign C = Slot3
  Forward check: remove Slot3 from D; D domain = ['Slot1']
  Forward check: remove Slot3 from E; E domain = ['Slot1', 'Slot2']
Try D = Slot1
```

```
 Assign D = Slot1
 Forward check: remove Slot1 from E; E domain = ['Slot2']
Skip E = Slot1 (pruned from domain)
Try E = Slot2
 Assign E = Slot2

Forward Checking Result
=======================
A - Artificial Intelligence: Slot1
B - Database Systems: Slot2
C - Computer Networks: Slot3
D - Operating Systems: Slot1
E - Software Engineering: Slot2
Assignment attempts: 5
Backtracks: 0
```

CONCLUSION:

Forward checking cuts the number of attempts in this run from 9 to 5 because it spots some conflicts before those values are ever tried. For example, once A = Slot1 is assigned, Slot1 is removed from B's domain, and once C = Slot3 is assigned, Slot3 is removed from the domains of D and E.

Both methods find a valid timetable here without undoing any committed assignment. Forward checking is more efficient in this run because it avoids testing values that are already known to be impossible. In a problem where a domain becomes empty, it can also detect the dead end early and trigger backtracking sooner.